Projektarbeit im Modul
Fabrikautomatisierung und Robotik

Bildverarbeitungsgestützte Steuerung eines DENSO COBOTTA über Handgesten

Verfasser: Abhinav Sridhar

Inhaltsverzeichnis

1 Einleitung und Zielsetzung **3**

2 Grundlagen **3**

2.1 Handgestenerkennung mit MediaPipe 3

2.2 DENSO COBOTTA und b-CAP 3

2.3 Tippbetrieb am Teach Pendant 4

3 Systemkonzept **4**

3.1 Anforderungen 4

3.2 Architektur 4

3.3 Gestenkonzept 5

3.4 Zustandsautomat 6

3.5 Sicherheitskonzept 7

4 Umsetzung **7**

4.1 Entwicklungsumgebung und Softwarestruktur 7

4.2 Gestenerkennung 8

4.3 Steuerung 8

4.4 Sicherheitsschicht 9

4.5 Anbindung des COBOTTA über b-CAP 9

4.6 Konfiguration und Protokollierung 9

5 Test und Auswertung **10**

5.1 Automatisierte Tests der Steuerung 10

5.2 Versuchsaufbau zur Gestenerkennung 10

5.3 Erkennungsrate 10

5.4 Auslösung auf Befehlsebene 11

5.5 Rechenzeit und Bildrate 12

5.6 Fehlauslösungen bei freier Handbewegung 12

5.7 Erprobung in der Simulation 12

6 Eigener Gestenklassifikator **14**

6.1 Ziel und Vorgehen 14

6.2 Datensatz 14

6.3 Merkmale 15

6.4 Verfahren und Validierung 16

6.5 Ergebnis auf der Testsitzung und Vergleich mit MediaPipe 16
6.6 Bewertung und Grenzen 17

7 Inbetriebnahme und Bedienung **17**
7.1 Einrichtung 17
7.2 Programme 17
7.3 Erprobung ohne Roboter: Simulation (VRC) 18
7.4 Anbindung an einen realen COBOTTA 18
7.5 Bedienung 18

8 Hinweise zur Weiterentwicklung **18**

9 Fazit und Ausblick **19**

A Konfigurationsdatei **20**

1 Einleitung und Zielsetzung

Industrieroboter werden üblicherweise über ein Handbediengerät (Teach Pendant) oder über Programme gesteuert. Für das schnelle Positionieren oder für Demonstrationen ist eine berührungslose Bedienung interessant: Der Bediener steuert den Roboter mit Handgesten vor einer Kamera, ohne ein Gerät in der Hand zu halten.

Ziel dieser Projektarbeit ist ein System, das Handgesten über eine Webcam erkennt und in sichere Bewegungsbefehle für einen DENSO COBOTTA umsetzt. Im Einzelnen:

- Erkennung von Handgesten in Echtzeit mit der Bibliothek MediaPipe,
- Umsetzung der Gesten in Roboterbefehle nach dem Prinzip des Tippbetriebs am Teach Pendant,
- ein Sicherheitskonzept mit Freigabe, priorisiertem Stopp und Arbeitsraumgrenzen,
- Ansteuerung des realen COBOTTA über die Schnittstelle b-CAP aus Python,
- eine quantitative Auswertung der Gestenerkennung,
- ein eigener, auf selbst aufgenommenen Daten trainierter Gestenklassifikator im Vergleich zum vortrainierten MediaPipe-Modell sowie
- eine Dokumentation, mit der nachfolgende Studierende das System in Betrieb nehmen und weiterentwickeln können.

Die Software wurde so aufgebaut, dass sie ohne Roboter vollständig entwickelt und getestet werden kann (simulierter Roboter) und für den realen Roboter nur das Roboter-Backend umgeschaltet wird.

2 Grundlagen

2.1 Handgestenerkennung mit MediaPipe

MediaPipe ist eine quelloffene Bibliothek von Google für maschinelles Sehen. Der verwendete *Gesture Recognizer* arbeitet in drei Stufen: Zuerst wird eine Hand im Bild gefunden, dann werden 21 Handpunkte (Landmarks, z. B. Fingerspitzen, Gelenke, Handgelenk) bestimmt, und schließlich ordnet ein vortrainiertes Klassifikationsmodell die Handhaltung einer von sieben Gesten zu: *Open_Palm*, *Closed_Fist*, *Pointing_Up*, *Thumb_Up*, *Thumb_Down*, *Victory* und *ILoveYou*. Erkennt das Modell keine dieser Gesten, liefert es *None*. Zu jeder Geste wird eine Konfidenz zwischen 0 und 1 ausgegeben.

Da das Modell bereits auf großen Datensätzen trainiert ist, ist für den Grundbetrieb kein eigenes Training nötig. Die dritte Stufe lässt sich jedoch durch ein eigenes Modell ersetzen, das die 21 Handpunkte als Eingabe verwendet (Abschnitt 6). Die Erkennung läuft lokal auf dem Rechner, ohne Internetverbindung.

2.2 DENSO COBOTTA und b-CAP

Der COBOTTA ist ein kompakter sechsachsiger Roboter von DENSO WAVE mit einer Reichweite von etwa 340 mm. Er wird von einer RC8-Steuerung betrieben. Für die Ansteuerung von außen stellt DENSO das Protokoll *b-CAP* bereit, ein binäres Protokoll über TCP/IP (hier Port 5007). Über b-CAP können Programme Verbindungen zur Steuerung

aufbauen, die aktuelle Position lesen und Bewegungsbefehle senden. DENSO stellt dafür einen quelloffenen Python-Client unter MIT-Lizenz bereit [2], der in diesem Projekt verwendet wird.

2.3 Tippbetrieb am Teach Pendant

Beim manuellen Verfahren eines Industrieroboters (Tippbetrieb) wählt der Bediener am Teach Pendant zunächst ein Koordinatensystem bzw. eine Achse und verfährt dann mit Plus- und Minus-Tasten in kleinen Schritten oder kontinuierlich, bei reduzierter Geschwindigkeit. Die Schrittweite (Inkrement) ist einstellbar. Dieses bekannte Bedienprinzip wurde auf die Gestensteuerung übertragen.

3 Systemkonzept

3.1 Anforderungen

Nr.	Anforderung
A1	Der Roboter lässt sich in X, Y und Z mit Handgesten verfahren.
A2	Ein Stopp ist jederzeit mit einer einfachen Geste möglich und hat Vorrang vor allen anderen Befehlen.
A3	Ohne bewusste Freigabe durch den Bediener bewegt sich der Roboter nicht.
A4	Verlässt die Hand das Kamerabild, stoppt das System.
A5	Der Roboter verlässt einen festgelegten Arbeitsraum nicht.
A6	Unsichere oder kurzzeitige Erkennungen lösen keine Befehle aus.
A7	Entwicklung und Test sind ohne Roboter möglich; für den realen Roboter wird nur das Roboter-Backend umgeschaltet.
A8	Alle Parameter sind ohne Programmänderung in einer Konfigurationsdatei einstellbar.

Tabelle 1: Anforderungen an das System

3.2 Architektur

Das System besteht aus einer Verarbeitungskette mit klar getrennten Komponenten (Abbildung 1). Jede Komponente hat genau eine Aufgabe und eine eigene Datei.

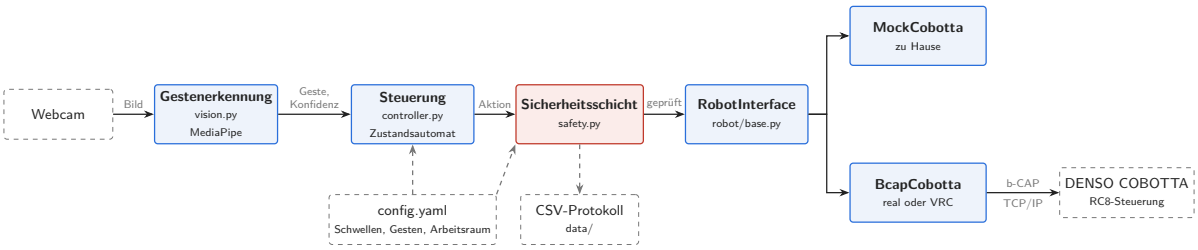


Abbildung 1: Softwarearchitektur: Verarbeitungskette von der Kamera bis zum Roboter

1. **Gestenerkennung** (`src/vision.py`): liest das Kamerabild und liefert je Bild die erkannte Geste und ihre Konfidenz.

2. **Steuerung** (`src/controller.py`): ein Zustandsautomat, der aus der Folge erkannter Gesten Aktionen erzeugt (Abschnitt 3.4).
3. **Sicherheitsschicht** (`src/safety.py`): prüft jede Aktion unabhängig von der Steuerung, bevor sie den Roboter erreicht.
4. **Roboter-Schnittstelle** (`src/robot/base.py`): eine abstrakte Klasse mit den Methoden `connect`, `get_position`, `move_relative`, `gripper`, `stop` und `disconnect`. Sie wird von einem simulierten Roboter (`MockCobotta`) und vom realen COBOTTA über b-CAP (`BcapCobotta`) implementiert.

Durch die abstrakte Schnittstelle kann der Roboter ausgetauscht werden, ohne Gestenerkennung, Steuerung oder Sicherheitslogik zu ändern. Welcher Roboter verwendet wird, legt allein der Eintrag `roboter.backend` in `config.yaml` fest.

3.3 Gestenkonzept

Geste	MediaPipe-Name	Befehl
Offene Hand	Open_Palm	STOPP (höchste Priorität)
Faust (halten)	Closed_Fist	Freigabe nach Start und nach jedem Stopp; sonst keine Aktion
Zeigefinger oben	Pointing_Up	aktive Achse wechseln X $\rightarrow$ Y $\rightarrow$ Z
Daumen hoch	Thumb_Up	ein Schritt in positiver Richtung der aktiven Achse
Daumen runter	Thumb_Down	ein Schritt in negativer Richtung der aktiven Achse
I-love-you-Zeichen	ILoveYou	Schrittweite umschalten 5 mm $\leftrightarrow$ 1 mm
Victory-Zeichen	Victory	Greifer öffnen/schließen

Tabelle 2: Zuordnung der Gesten zu Befehlen

Begründung der Gestaltung:

- **Achse wählen, dann verfahren:** entspricht dem Tippbetrieb am Teach Pendant und ist damit für Anwender aus der Robotik vertraut.
- **Nur vortrainierte Gesten:** Die sieben MediaPipe-Gesten sind auf großen Datensätzen trainiert und daher robust. Eine Steuerung mit je einer Geste pro Achsrichtung würde neun Gesten erfordern und damit ein eigenes Modell sowie mehr Verwechslungsmöglichkeiten.
- **Diskrete Schritte statt Handposition:** Eine Steuerung über die Position der Hand im Bild wäre intuitiver, aber ungenau (Tiefe nur schwer bestimmbar) und würde jede unbeabsichtigte Handbewegung an den Roboter weitergeben. Feste Schritte sind vorhersehbar.
- **Offene Hand als Stopp:** allgemein verständliches Haltezeichen.
- **Faust halten als Freigabe:** eine bewusste Handlung, die nicht versehentlich entsteht.
- **Achsfarben:** Die aktive Achse wird in der in Robotik und CAD üblichen Farbkonvention angezeigt: X rot, Y grün, Z blau.

- **Grob- und Feinschritt:** 5 mm zum zügigen Annähern, 1 mm zum genauen Positionieren, entsprechend dem einstellbaren Inkrement am Teach Pendant.

3.4 Zustandsautomat

Die Steuerung kennt zwei Zustände (Abbildung 2). Nach dem Start befindet sie sich im Zustand GESTOPPT; Bewegungen sind erst nach einer Freigabe möglich.

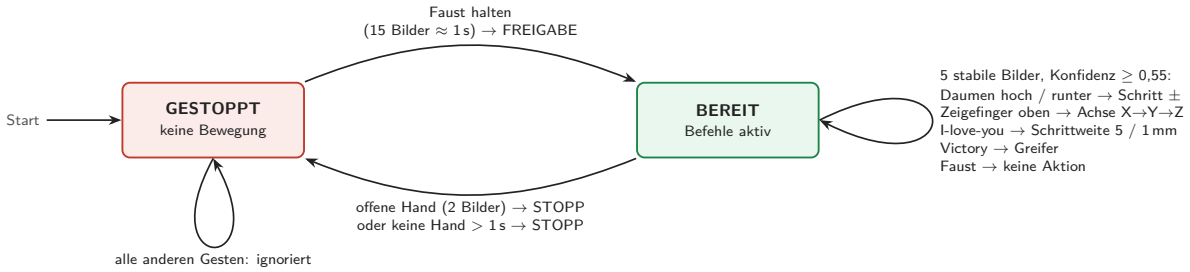


Abbildung 2: Zustandsautomat der Steuerung

Damit Erkennungsfehler einzelner Bilder keine Befehle auslösen, wird ein Befehl erst ausgeführt, wenn dieselbe Geste in mehreren aufeinanderfolgenden Bildern mit ausreichender Konfidenz erkannt wurde (Entprellung). Zwei Arten von Befehlen werden unterschieden:

- **Tippbetrieb:** Schritt plus/minus wiederholt sich beim Halten der Geste im Takt der Sperrzeit (0,5 s). Eine kurz gezeigte Geste erzeugt genau einen Schritt.
- **Flankengesteuert:** Achswechsel, Schrittweite und Greifer werden je gezeigter Geste nur einmal ausgeführt, auch wenn die Geste länger gehalten wird. So springt die Achse beim Halten nicht unkontrolliert weiter.

3.5 Sicherheitskonzept

Maßnahme	Umsetzung und Begründung
Start im Zustand GESTOPPT	Nach dem Programmstart ist keine Bewegung möglich (A3).
Freigabe durch Faust	Faust ca. 1 s halten (15 Bilder). Bewusste Handlung, entsteht nicht zufällig.
Priorisierter Stopp	Offene Hand stoppt nach 2 Bildern, in jedem Zustand und vor allen anderen Befehlen (A2).
Niedrigere Schwelle für Stopp (fail-safe)	Stopp ab Konfidenz 0,45, alle anderen Befehle erst ab 0,55. Ein unnötiger Stopp ist harmlos, ein verpasster Stopp gefährlich.
Quittierung	Nach jedem Stopp ist eine erneute Freigabe nötig.
Überwachung der Hand	Ist länger als 1 s keine Hand sichtbar, stoppt das System (A4).
Entprellung	Befehle erst nach 5 aufeinanderfolgenden Bildern mit Konfidenz $\geq 0{,}55$ (A6).
Sicherheitsschicht (Redundanz)	Prüft unabhängig von der Steuerung: Freigabe vorhanden, Schritt ≤ 10 mm, Zielposition im Arbeitsraum (A5). Unzulässige Befehle werden abgelehnt und protokolliert.
Reduzierte Geschwindigkeit	Der reale Roboter fährt mit 10 % der Geschwindigkeit.
Hardware-Not-Halt	Bleibt jederzeit wirksam und ist der Software übergeordnet.

Tabelle 3: Maßnahmen des Sicherheitskonzepts

Das Sicherheitskonzept ist mehrstufig: Steuerung und Sicherheitsschicht prüfen unabhängig voneinander, und die Sicherheitsfunktionen des Roboters selbst (Not-Halt, Kraftbegrenzung des kollaborierenden Roboters) bleiben unberührt. Es handelt sich um eine Software-Absicherung für den Versuchs- und Demonstrationsbetrieb, nicht um eine zertifizierte Sicherheitsfunktion im Sinne der Maschinensicherheit.

4 Umsetzung

4.1 Entwicklungsumgebung und Softwarestruktur

Die Software ist in Python 3.11 geschrieben und wurde auf einem MacBook Air (Apple M3) mit der eingebauten Webcam entwickelt. Verwendete Bibliotheken: MediaPipe (Gestenerkennung), OpenCV (Kamera und Anzeige), PyYAML (Konfiguration), Matplotlib (Diagramme), scikit-learn (eigener Klassifikator) sowie der b-CAP-Client von DENSO WAVE.

Datei	Aufgabe
`main.py`	Hauptprogramm: verbindet alle Komponenten, Anzeige, Protokoll
`gesture_demo.py`	nur Gestenerkennung, ohne Roboter
`config.yaml`	alle Einstellungen
`src/vision.py`	Kamera und MediaPipe-Gestenerkennung
`src/klassifikator.py`	Merkmale und eigener Gestenklassifikator
`src/controller.py`	Zustandsautomat: Geste → Aktion
`src/safety.py`	Sicherheitsschicht
`src/robot/base.py`	abstrakte Roboter-Schnittstelle
`src/robot/mock.py`	simulierter COBOTTA
`src/robot/bcap_cobotta.py`	realer COBOTTA über b-CAP
`third_party/bCAPClient/`	b-CAP-Client von DENSO WAVE (MIT-Lizenz)
`scripts/test_steuerung.py`	automatischer Test von Steuerung und Sicherheitsschicht
`scripts/test_bcap_verbindung.py`	schrittweiser Verbindungstest (Simulation oder realer Roboter)
`scripts/aufnahme_erkennung.py`	geführte Aufnahme für die Auswertung
`scripts/analyse_erkennung.py`	Auswertung der Gestenerkennung
`scripts/analyse_protokoll.py`	Auswertung der Protokolle von `main.py`
`scripts/aufnahme_datensatz.py`	Datensatz für den eigenen Klassifikator aufnehmen
`scripts/training_klassifikator.py`	Training, Validierung, Vergleich mit MediaPipe
`scripts/vergleich_merkmale.py`	Vergleich Rohkoordinaten / Formmerkmale
`scripts/test_klassifikator.py`	automatischer Test der Merkmalsberechnung

Tabelle 4: Dateien des Projekts

4.2 Gestenerkennung

Der MediaPipe Gesture Recognizer wird im Video-Modus mit einer Hand betrieben. Jedes Kamerabild wird gespiegelt (natürlicheres Gestikulieren, wie vor einem Spiegel) und mit einem streng steigenden Zeitstempel an das Modell übergeben. Da die Kamera unter macOS beim Start oft einige leere Bilder liefert, wartet die Kamera-Klasse bis zu 2 s auf gültige Bilder und gibt sonst eine Fehlermeldung mit möglichen Ursachen aus.

Über `erkennung.backend` in `config.yaml` wird festgelegt, wer die Geste zuordnet: der vortrainierte MediaPipe-Klassifikator (`mediapipe`, Standard) oder das eigene Modell (`eigen`, Abschnitt 6). Beide liefern dieselben Gestennamen und eine Konfidenz, sodass Steuerung und Sicherheitsschicht unverändert bleiben.

4.3 Steuerung

Die Klasse `GestureController` erhält je Bild die erkannte Geste, die Konfidenz und die Zeit und gibt eine Liste von Aktionen zurück (`FREIGABE`, `STOPP`, `BEWEGEN`, `ACHSE`, `SCHRITTWEITE`, `GREIFER`). Sie zählt, in wie vielen aufeinanderfolgenden Bildern dieselbe Geste über der Schwelle erkannt wurde, und vergleicht diesen Zähler mit den Grenzwerten aus der Konfiguration. Die Steuerung kennt keinen Roboter; sie erzeugt nur Aktionen. Dadurch lässt sie sich ohne Kamera und Roboter automatisch testen.

4.4 Sicherheitsschicht

Die Klasse `SafetyLayer` führt Aktionen nur aus, wenn sie zulässig sind. Für eine Bewegung liest sie die aktuelle Position vom Roboter, berechnet die Zielposition und prüft sie gegen die Arbeitsraumgrenzen. Jede Aktion liefert ein Ergebnis (ausgeführt/abgelehnt) und eine Meldung, die angezeigt und protokolliert wird.

4.5 Anbindung des COBOTTA über b-CAP

Die Klasse `BcapCobotta` verwendet den Python-Client von DENSO WAVE. Ablauf:

1. TCP-Verbindung zur RC8-Steuerung (IP-Adresse 192.168.0.1, Port 5007, laut Lehrveranstaltungsprojekt),
2. Controller-Verbindung (Provider `CaoProv.DENSO.VRC` wie im Python-Beispiel von DENSO, einstellbar in `config.yaml`) und Roboter-Objekt `Arm`,
3. Fehler quittieren, Armberechtigung holen (`TakeArm`), Motor einschalten,
4. reduzierte Geschwindigkeit setzen (`ExtSpeed`, 10 %),
5. Position über die Variable `@CURRENT_POSITION` lesen,
6. Schritte als Linearbewegung zu einer Zielpose `P(X, Y, Z, Rx, Ry, Rz, Fig)` = aktuelle Pose + Schritt, mit unveränderter Orientierung,
7. Stopp über `Halt`; beim Beenden Motor aus und Armberechtigung abgeben.

Bewegt wird mit der DENSO-Bewegungsoption `@E` (einstellbar über `bewegung.bahnende`): Der Aufruf kehrt erst zurück, wenn der Roboter das Ziel erreicht hat. Die Schritte werden also synchron ausgeführt. In der Simulation dauert ein 5-mm-Schritt bei 10 % Geschwindigkeit 0,58 s, ein 1-mm-Schritt 0,26 s (Abschnitt 5.7).

Erfahrungen aus der Inbetriebnahme in der Simulation. Die Anbindung wurde mit der virtuellen Robotersteuerung VRC von DENSO erprobt (Abschnitt 5.7). Dabei ergaben sich zwei Korrekturen am Code:

- **Provider:** Zunächst war `CaoProv.DENSO.RC8` eingetragen. Das Python-Beispiel von DENSO für eine direkte b-CAP-Verbindung verwendet jedoch `CaoProv.DENSO.VRC`; damit gelang die Verbindung auf Anhieb. Der Wert steht jetzt in `config.yaml`.
- **Bewegungsoption:** Mit der zunächst verwendeten Option `@P` (Überschleifen) meldete die Steuerung den Schritt bereits als beendet, als der Roboter erst 2,9 der 5 mm zurückgelegt hatte. Mit `@E` wurden 4,998 mm gemessen, nach dem Rückweg betrug die Abweichung zur Startposition 0,000 mm.

4.6 Konfiguration und Protokollierung

Alle Einstellungen stehen in `config.yaml` (Anhang A): Kamera, Konfidenzschwellen, Entprellung, Zuordnung der Gesten, Schrittweiten, Geschwindigkeit, Arbeitsraum, Roboterverbindung und Greifer. Jeder Lauf von `main.py` wird in `data/protokoll_<datum_uhrzeit>.csv` protokolliert: Zeit, Geste, Konfidenz, Zustand, Aktion, Ergebnis, Meldung, Reaktionszeit, Ausführungsdauer und Position.

5 Test und Auswertung

5.1 Automatisierte Tests der Steuerung

Das Skript `scripts/test_steuerung.py` spielt simulierte Gestenfolgen mit 30 Bildern pro Sekunde durch Steuerung und Sicherheitsschicht und prüft das Verhalten gegen das Sicherheitskonzept. Alle 21 Prüfungen sind bestanden, unter anderem:

- Start im Zustand GESTOPPT; ohne Freigabe keine Bewegung,
- kurze Faust (0,2 s) gibt nicht frei; gehaltene Faust gibt frei,
- zu kurze Gesten (0,1 s) und Bewegungsgesten mit Konfidenz 0,5 lösen nichts aus,
- Achswechsel, Schrittweite und Greifer schalten nur einmal pro Geste,
- Stopp nach 0,1 s, auch bei Konfidenz 0,5; danach keine Bewegung ohne neue Freigabe,
- kein Stopp bei 0,8 s ohne Hand, automatischer Stopp nach mehr als 1 s,
- die Arbeitsraumgrenze wird nicht überschritten; Schritte darüber hinaus werden abgelehnt.

5.2 Versuchsaufbau zur Gestenerkennung

Mit `scripts/aufnahme_erkennung.py` wurde jede der sieben Gesten jeweils 2 s lang aufgenommen und jedes Bild protokolliert (erkannte Geste, Konfidenz, Rechenzeit). Die Aufnahmen erfolgten mit der eingebauten Webcam eines MacBook Air (M3) unter vier Bedingungen: helles bzw. gedimmtes Licht, jeweils mit 50 cm und 100 cm Abstand der Hand zur Kamera. Die Bedingung „hell, 50 cm“ wurde wiederholt (siehe unten). Die Videoeffekte von macOS (Reaktionen, Center Stage, Porträt) waren ausgeschaltet, da sie direkt ins Kamerabild gezeichnet werden. Die Gesten-Aufnahmen umfassen insgesamt 1416 Bilder. Zusätzlich wurden 60 s freie Handbewegung ohne beabsichtigte Befehlsgesten aufgenommen (1027 Bilder).

Als „richtig erkannt“ gilt ein Bild, wenn MediaPipe die gezeigte Geste mit einer Konfidenz über der jeweiligen Schwelle erkennt (0,55 bzw. 0,45 für den Stopp).

5.3 Erkennungsrate

Geste	Bilder	Erkennungsrate	mittlere Konfidenz
Offene Hand	181	100,0 %	0,62
Zeigefinger oben	180	96,7 %	0,80
Daumen hoch	211	100,0 %	0,67
Daumen runter	211	100,0 %	0,91
Victory	212	100,0 %	0,86
Faust	210	82,4 %	0,81
I love you	211	100,0 %	0,76
gesamt	1416	**97,0 %**	

Tabelle 5: Erkennungsrate je Geste über alle Bedingungen (mittlere Konfidenz der richtig erkannten Bilder)

Geste	dunkel 100 cm	dunkel 50 cm	hell 100 cm	hell 50 cm	hell 50 cm (Wdh.)
Offene Hand	100,0 %	100,0 %	100,0 %	100,0 %	100,0 %
Zeigefinger oben	100,0 %	100,0 %	83,9 %	100,0 %	98,3 %
Daumen hoch	100,0 %	100,0 %	100,0 %	100,0 %	100,0 %
Daumen runter	100,0 %	100,0 %	100,0 %	100,0 %	100,0 %
Victory	100,0 %	100,0 %	100,0 %	100,0 %	100,0 %
Faust	100,0 %	100,0 %	100,0 %	38,3 %	100,0 %
I love you	100,0 %	100,0 %	100,0 %	100,0 %	100,0 %

Tabelle 6: Erkennungsrate je Geste und Aufnahmebedingung

Fünf der sieben Gesten wurden unter allen Bedingungen in jedem Bild erkannt. Die Faust erreichte in der ersten Aufnahme „hell, 50 cm“ nur 38,3 %; in den übrigen Fällen wurde sie nicht als andere Geste, sondern als „keine Geste“ eingestuft. Die Wiederholung unter gleichen Bedingungen ergab 100 % bei einer mittleren Konfidenz von 0,90. Die Ursache lag daher wahrscheinlich in der Handhaltung der ersten Aufnahme (zu nah an der Kamera oder schräg gehalten), nicht im System. Beide Aufnahmen gehen in die Gesamtwerte ein.

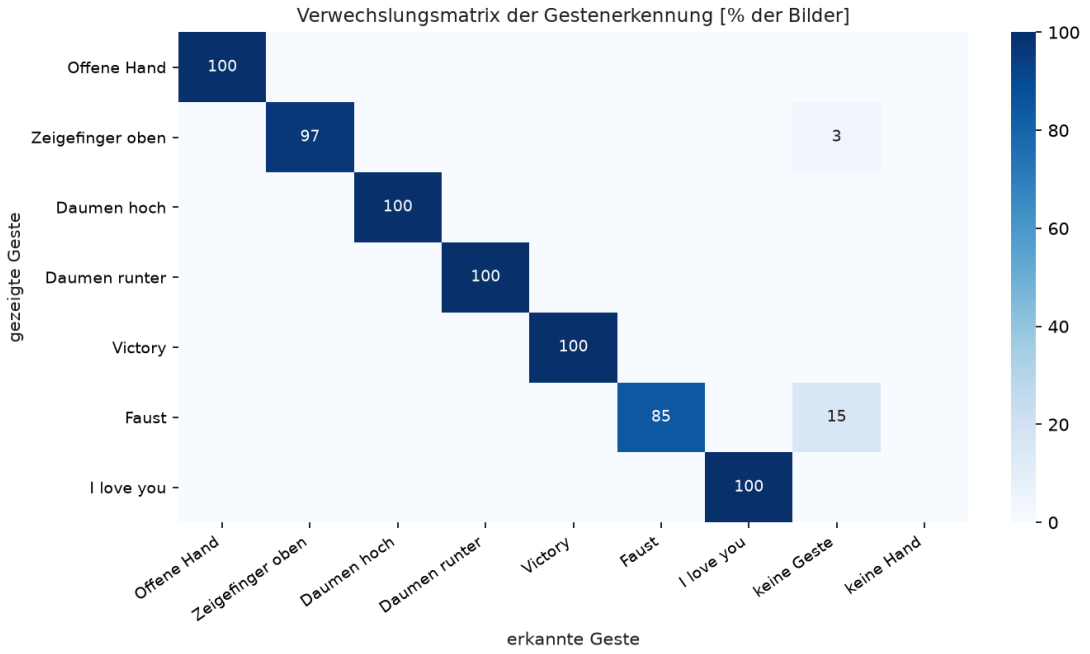


Abbildung 3: Verwechslungsmatrix: Anteil der Bilder je gezeigter (Zeile) und erkannter Geste (Spalte)

Die Verwechslungsmatrix (Abbildung 3) zeigt das wichtigste Ergebnis für die Sicherheit: **Keine Geste wurde als eine andere Geste erkannt.** Alle 43 nicht richtig erkannten Bilder waren „keine Geste“, „keine Hand“ oder die richtige Geste knapp unter der Schwelle. Ein Erkennungsfehler führt damit zu *keiner* Aktion, nicht zu einer falschen.

5.4 Auslösung auf Befehlsebene

Jede Aufnahme wurde zusätzlich durch den Zustandsautomaten gespielt. Ausgewertet wurde, ob der richtige Befehl ausgelöst wird und nach welcher Zeit. Die Zeit zählt ab dem ersten

aufgenommenen Bild, in dem die Geste bereits gezeigt wird, und entspricht damit der Entprellzeit der Steuerung.

Geste	richtiger Befehl	mittlere Auslösezeit	falsche Befehle
Offene Hand (Stopp)	5 / 5	60 ms	0
Zeigefinger oben	5 / 5	239 ms	0
Daumen hoch	5 / 5	213 ms	0
Daumen runter	5 / 5	213 ms	0
Victory	5 / 5	214 ms	0
Faust (Freigabe)	4 / 5	816 ms	0
I love you	5 / 5	214 ms	0

Tabelle 7: Auslösung der Befehle bei Simulation der Aufnahmen durch die Steuerung

In 34 von 35 Aufnahmen wurde der richtige Befehl ausgelöst, falsche Befehle traten nicht auf. Die fehlende Freigabe betrifft die oben beschriebene erste Faust-Aufnahme. Der Stopp reagiert wie beabsichtigt deutlich schneller als alle anderen Befehle.

5.5 Rechenzeit und Bildrate

Die Gestenerkennung benötigt im Mittel 12,9 ms pro Bild (Median 14,1 ms, 95-%-Quantil 14,9 ms, Maximum 27,9 ms). Die Kamera liefert dagegen nur 15 Bilder pro Sekunde (Median; bis 30 Bilder pro Sekunde bei hellem Licht), da die Belichtungsautomatik bei wenig Licht die Bildrate senkt. Die Reaktionszeit wird also durch die Kamera begrenzt, nicht durch die Erkennung. Bei 15 Bildern pro Sekunde ergeben sich Entprellzeiten von etwa 270 ms für Bewegungsbefehle, 70 ms für den Stopp und 0,9 s für die Freigabe.

5.6 Fehlauslösungen bei freier Handbewegung

Während 60 s natürlicher Handbewegung (Gestikulieren, Gesicht berühren, Tippen) wurde im ungünstigsten Fall, also bei durchgehend freigegebenem System, **kein einziger Bewegungsbefehl** und kein sonstiger Befehl ausgelöst. Es traten 16 Stopps auf, da eine offene Hand beim Gestikulieren häufig vorkommt. Diese Stopps sind harmlos und die bewusste Folge der niedrigen Stopp-Schwelle (fail-safe): Das System stoppt lieber einmal zu oft als einmal zu wenig.

5.7 Erprobung in der Simulation

Die vollständige Kette – Kamera, Erkennung, Steuerung, Sicherheitsschicht und b-CAP-Anbindung – wurde mit der virtuellen Robotersteuerung VRC von DENSO erprobt. Die Simulation läuft in einer Windows-11-VM (UTM) auf dem MacBook, mit dem Beispielprojekt `Cobotta_test.WPJ` (COBOTTA CVR038A1). Als b-CAP-Server dient der *bCapListener* aus ORiN2. `main.py` läuft unverändert auf dem Mac mit der Webcam und verbindet sich über das virtuelle Netzwerk mit der VM (Abbildung 4). Gegenüber dem realen Roboter ändern sich nur die IP-Adresse und das Roboter-Backend (`--roboter bcap --ip <IP der VM>`).

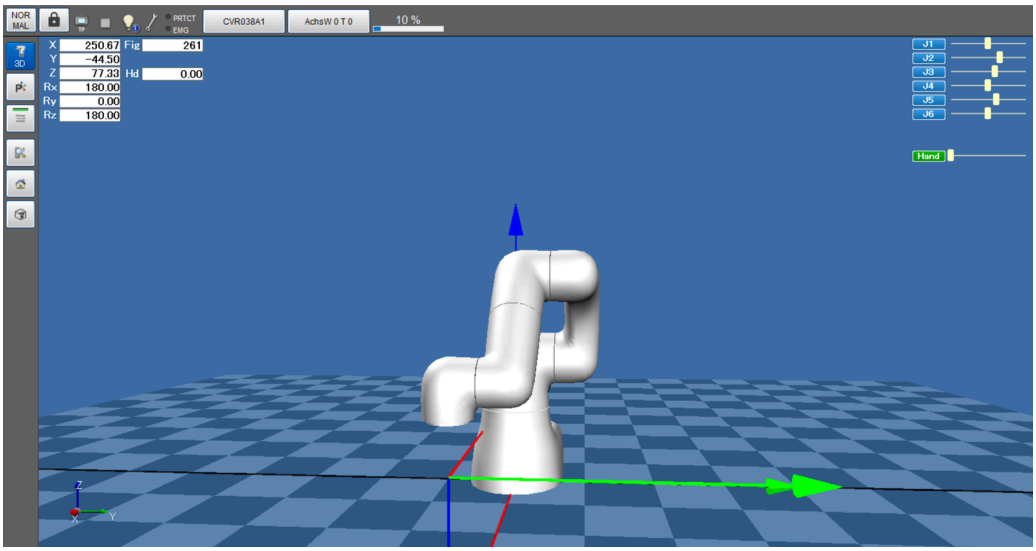


Abbildung 4: Simulierter COBOTTA in der virtuellen Steuerung VRC (virtuelles Teach Pendant mit 3D-Ansicht)

Verbindungstest (`scripts/test_bcap_verbindung.py`): Verbindung, Lesen der Position und Halt funktionierten. Ein Schritt von 5 mm in Z ergab 4,998 mm in 0,59 s; nach dem Rückweg lag die Abweichung zur Startposition bei 0,000 mm.

Gestensteuerung: In zwei Läufen von zusammen 7,3 min wurden 322 Aktionen ausgeführt und mit `scripts/analyse_protokoll.py` ausgewertet (`docs/auswertung/ergebnis_simulation.md`).

Aktion	Anzahl (ausgeführt)	Auslösezeit (Median)	Ausführung (Median)
Freigabe (Faust)	14 (14)	468 ms	–
Stopp (offene Hand)	11 (11)	35 ms	0,7 ms
Stopp (keine Hand sichtbar)	7 (7)	978 ms	0,8 ms
Schritt 5 mm / 1 mm	269 (236)	167 ms	583 ms / 263 ms
Achse wechseln	11 (11)	265 ms	–
Schrittweite umschalten	8 (8)	268 ms	–

Tabelle 8: Gestensteuerung des simulierten COBOTTA (10 % Geschwindigkeit). Auslösezeit: erstes Bild der Geste bis zum Befehl; beim Stopp ohne Hand planmäßig nach der Wartezeit von 1 s.

Ergebnisse:

- **Schrittgenauigkeit:** Alle 236 ausgeführten Schritte entsprachen der Soll-Schrittweite (Abweichung 0,0 mm bei 0,1 mm Auflösung des Protokolls). Abgedeckt waren alle drei Achsen in beiden Richtungen mit beiden Schrittweiten (12 Kombinationen).
- **Arbeitsraum:** 33 Schritte wurden von der Sicherheitsschicht abgelehnt, weil sie die Grenzen $X = 300$, $Y = 150$ bzw. $Z = 250$ mm überschritten hätten. Der Roboter blieb an der Grenze stehen.
- **Keine Fehlbefehle:** Bei bewusst ungeordneter Handbewegung zwischen den Gesten (5 Versuche) wurde kein Befehl ausgelöst.
- **Stopp während eines Schritts:** Da die Schritte synchron ausgeführt werden, wertet das Programm während eines Schritts keine Kamerabilder aus. Ein Stopp wirkt daher

erst nach dem laufenden Schritt: im ungünstigsten Fall nach der Restdauer des Schritts (bis 0,58 s bei 5 mm, bis 0,26 s bei 1 mm) plus etwa 35 ms. Nach dem letzten Schritt bewegte sich der Roboter in allen Versuchen nicht weiter, auch wenn die offene Hand erst einige Sekunden später erkannt wurde.

Nicht abgedeckt: der Greifer (in `config.yaml` deaktiviert, es wurde nur die Logik geprüft), die unteren Arbeitsraumgrenzen sowie die Ende-zu-Ende-Latenz bis zur tatsächlichen Bewegung, die nur am realen Roboter mit einer Zeitlupenaufnahme messbar ist. Die Arbeitsraumgrenzen sind Beispielwerte und müssen vor einem Einsatz am realen Roboter an den Aufbau angepasst werden.

6 Eigener Gestenklassifikator

6.1 Ziel und Vorgehen

Zusätzlich zum vortrainierten MediaPipe-Klassifikator wurde ein eigenes Modell entwickelt. MediaPipe liefert weiterhin die Hand und ihre 21 Punkte; nur die Zuordnung zu einer Geste übernimmt das eigene Modell. Ziel war, den vollständigen Ablauf des überwachten Lernens selbst umzusetzen (Datenerhebung, Merkmale, Vergleich mehrerer Verfahren, Validierung, unabhängiger Test) und das Ergebnis messbar mit MediaPipe zu vergleichen.

Das Modell unterscheidet acht Klassen: die sieben Befehlsgesten und „keine Befehlsgeste“. Die achte Klasse ist notwendig, da ein Klassifikator sonst jede Handhaltung, z. B. eine locker gehaltene Hand, der ähnlichsten Befehlsgeste zuordnen würde. Sie wird wie *None* bei MediaPipe behandelt und löst keine Aktion aus. Als Konfidenz dient die Klassenwahrscheinlichkeit des Modells; es gelten dieselben Schwellen wie für MediaPipe.

6.2 Datensatz

Mit `scripts/aufnahme_datensatz.py` wurde jede Klasse 10 s lang aufgenommen. Gespeichert werden nur die 21 Handpunkte (MediaPipe-Weltkoordinaten in Metern), keine Bilder, sowie zum Vergleich das Ergebnis des MediaPipe-Klassifikators. Während der Aufnahme wurde die Geste gehalten, die Hand aber langsam gedreht, geneigt und in Abstand und Lage verändert. Für „keine Befehlsgeste“ wurden lockere und halb geöffnete Hände, Übergänge zwischen Gesten und andere Fingerstellungen gezeigt.

Sitzung	Verwendung	Bilder mit Hand
hell, ca. 50 cm	Training	2366
hell, ca. 100 cm	Training	2382
gedimmtes Licht	Training	2380
anderer Zeitpunkt und Ort	Test	728
gesamt		7856

Tabelle 9: Datensatz des eigenen Klassifikators (je Klasse etwa 300 Bilder pro Trainingssitzung, in der Testsitzung 6 s bzw. etwa 90 Bilder je Klasse)

Die Testsitzung wurde zu einem anderen Zeitpunkt an einem anderen Ort aufgenommen und ausschließlich für die abschließende Bewertung verwendet. Das mitgespeicherte

MediaPipe-Ergebnis diente als Plausibilitätsprüfung: In einer ersten Aufnahme erkannte MediaPipe bei der Klasse „Zeigefinger oben“ in allen 299 Bildern „Daumen hoch“. Es war also die falsche Geste gezeigt worden; die Aufnahme wurde verworfen und wiederholt.

6.3 Merkmale

Erster Ansatz waren die Koordinaten der Handpunkte selbst: Ursprung in das Handgelenk verschoben und auf den Abstand Handgelenk–Mittelfingergrundgelenk skaliert, also 20 Punkte $\times$ 3 Koordinaten = 60 Werte. Innerhalb der Aufnahmen erreichten alle Verfahren damit einen Makro-F1 von 0,997. Wurde jedoch auf zwei Sitzungen trainiert und auf der dritten bewertet, sank der Wert deutlich (Abbildung 5). Die Analyse zeigte die Ursache: Das Modell hatte die Neigung der Hand zur Kamera (z-Koordinate) als Abkürzung gelernt. In den hellen Aufnahmen war die Hand beim Zeigefinger zufällig nach hinten, bei der Faust nach vorne geneigt. In der gedimmten Aufnahme war die zeigende Hand nach vorne geneigt und wurde in 281 von 283 Bildern als Faust eingestuft, obwohl der gestreckte Zeigefinger klar erkennbar war. Hinzu kommt, dass die Tiefe bei einer einzelnen Webcam der unsicherste Wert ist.

Daher beschreiben die endgültigen Merkmale die *Form* der Hand statt ihrer Lage im Raum (`src/klassifikator.py`):

- 15 Beugewinkel: je Finger die Winkel an den drei Gelenken (gestreckt $\approx 180°$),
- 5 Abstände der Fingerspitzen zum Handgelenk und 4 Abstände benachbarter Fingerspitzen, jeweils auf die Handgröße normiert,
- 2 + 2 Werte für die Richtung der Hand und des Daumens in der Bildebene (ohne Tiefe), damit Daumen hoch und Daumen runter unterscheidbar bleiben.

Winkel und Abstände ändern sich weder durch Verschieben, Skalieren noch durch Neigen der Hand; die linke Hand wird gespiegelt. Diese Eigenschaften werden von `scripts/test_klassifikator.py` automatisch geprüft (11 von 11 Prüfungen bestanden).

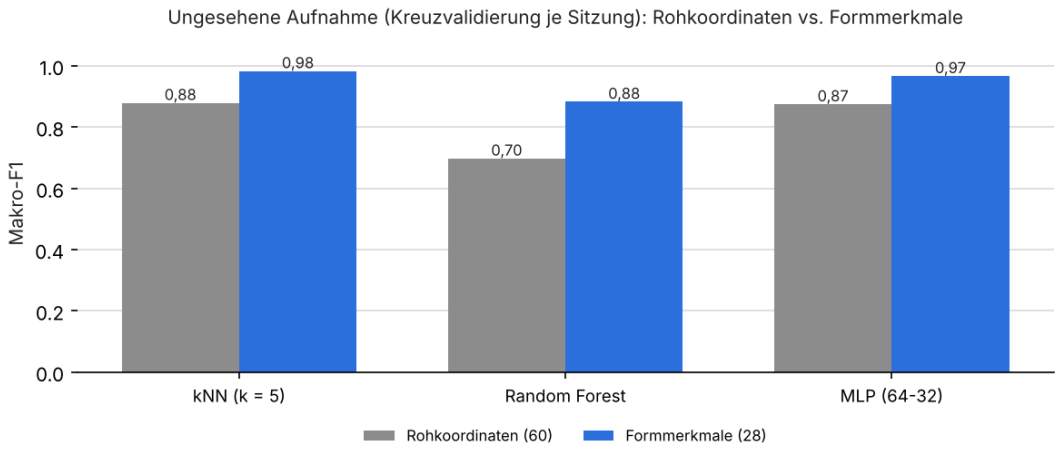


Abbildung 5: Makro-F1 auf jeweils ungesehener Trainingssitzung (Mittelwert über die drei Sitzungen): Rohkoordinaten im Vergleich zu Formmerkmalen

6.4 Verfahren und Validierung

Verglichen wurden drei Verfahren aus scikit-learn [4]: k-nächste Nachbarn (kNN, $k = 5$, mit Standardisierung), Random Forest (200 Bäume) und ein mehrschichtiges Perzeptron (MLP, zwei verdeckte Schichten mit 64 und 32 Neuronen, mit Standardisierung).

Aufeinanderfolgende Kamerabilder sind nahezu gleich. Eine zufällige Aufteilung in Trainings- und Testbilder würde daher fast identische Bilder in beiden Teilen enthalten und die Güte überschätzen. Bewertet wurde deshalb mit einer *Kreuzvalidierung je Sitzung* (Leave-one-session-out): Training auf zwei Sitzungen, Bewertung auf der dritten, für jede Sitzung einmal. Zum Vergleich wurde auch eine Kreuzvalidierung über 2-s-Abschnitte innerhalb der Aufnahmen berechnet.

	ausgelassene Sitzung				
Verfahren	gedimmt	hell 100 cm	hell 50 cm	**Mittelwert**	2-s-Abschnitte
kNN ($k = 5$)	0,997	0,978	0,972	**0,982**	0,994
Random Forest	0,996	0,845	0,806	0,882	0,997
MLP (64-32)	0,998	0,982	0,918	0,966	0,994

Tabelle 10: Makro-F1 der Kreuzvalidierung mit Formmerkmalen

Die Abschnitte innerhalb der Aufnahmen bewerten alle Verfahren nahezu gleich gut, die Bewertung je Sitzung zeigt dagegen deutliche Unterschiede. Der Random Forest passt sich am stärksten an die Handhaltung einer einzelnen Sitzung an und verallgemeinert am schlechtesten. Ausgewählt wurde anhand der Kreuzvalidierung je Sitzung (nicht anhand der Testsitzung) das kNN-Verfahren. Es benötigt auf dem MacBook Air (M3) 0,3 ms pro Bild, gegenüber etwa 13 ms für die MediaPipe-Verarbeitung insgesamt.

6.5 Ergebnis auf der Testsitzung und Vergleich mit MediaPipe

Die Testsitzung wurde von beiden Klassifikatoren mit denselben Bildern und denselben Schwellen ausgewertet, zusätzlich auf Steuerungsebene durch den Zustandsautomaten wie im Betrieb.

	Eigenes Modell (kNN)	MediaPipe
Genauigkeit (Bildebene)	100,0 %	99,7 %
Makro-F1	1,000	0,997
falsche Befehlsgeste (Anteil aller Bilder)	0,0 %	0,3 %
Befehlsgeste bei „keine Befehlsgeste“	0,0 %	2,2 %
richtiger Befehl ausgelöst (Steuerungsebene)	7 / 7	7 / 7
Befehle bei „keine Befehlsgeste“ (Steuerungsebene)	0	0

Tabelle 11: Testsitzung (728 Bilder): eigenes Modell und MediaPipe im Vergleich

Auf der Testsitzung ordnete das eigene Modell alle Bilder richtig zu. MediaPipe erkannte in zwei Bildern ohne Befehlsgeste den Zeigefinger; auf Steuerungsebene führte dies wegen der Entprellung zu keinem Befehl. Das MLP erreichte ebenfalls 100 %, der Random Forest nur 78,6 %.

6.6 Bewertung und Grenzen

- Das eigene Modell ist auf diesen Daten mindestens gleichwertig zu MediaPipe. Die Testsitzung war mit 6 s je Geste kurz und offenbar einfach (auch MediaPipe erreichte 99,7 %). Aussagekräftiger ist daher der Mittelwert der Kreuzvalidierung je Sitzung von 0,982.
- Das Modell wurde mit den Händen einer einzigen Person trainiert. MediaPipe ist auf vielen Personen trainiert und verallgemeinert daher voraussichtlich besser auf andere Bediener. Für einen allgemeinen Einsatz wären Aufnahmen weiterer Personen nötig.
- Bei der Kreuzvalidierung erkannte das MLP in einer Sitzung die offene Hand nur in etwa 60 % der Bilder. Da der Stopp bereits nach zwei Bildern auslöst, wäre der Stopp dennoch ausgelöst worden; ein verpasster Stopp ist aber der kritischste Fehler. Die Stopp-Erkennung sollte daher nicht von einem einzelnen Modell abhängen (siehe Ausblick).
- Der Hauptnutzen liegt in der Erweiterbarkeit: Eine neue Geste erfordert nur eine weitere Aufnahme und ein erneutes Training, während der MediaPipe-Klassifikator auf seine sieben Gesten festgelegt ist.

Standard bleibt der MediaPipe-Klassifikator, da er umfangreich erprobt ist; das eigene Modell wird über `erkennung.backend: eigen` aktiviert.

7 Inbetriebnahme und Bedienung

7.1 Einrichtung

Voraussetzung ist Python 3.11 (unter macOS z. B. über Homebrew). Im Projektordner:

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Beim ersten Start wird das MediaPipe-Modell (ca. 8 MB) automatisch nach `models/` heruntergeladen. Unter macOS muss dem Terminal der Kamerazugriff erlaubt werden.

7.2 Programme

```
python gesture_demo.py                  # nur Gestenerkennung
python main.py                          # Gestensteuerung (Roboter laut config.yaml)
python scripts/test_steuerung.py        # automatischer Test der Steuerung
python scripts/test_bcap_verbindung.py  # Verbindungstest (real: IP aus config.yaml)
python scripts/test_bcap_verbindung.py --ip <IP>   # Verbindungstest Simulation
python main.py --roboter bcap --ip <IP>            # Gestensteuerung Simulation
python scripts/analyse_protokoll.py data/protokoll_X.csv --titel "Simulation" --ausgabe
    ergebnis_simulation.md

# eigener Klassifikator
python scripts/aufnahme_datensatz.py --sitzung train_hell_50cm   # je Sitzung
python scripts/aufnahme_datensatz.py --sitzung test_1 --dauer 6  # Testsitzung
python scripts/training_klassifikator.py   # Training, Bericht, Modell
python scripts/vergleich_merkmale.py       # Rohkoordinaten vs. Formmerkmale
python scripts/test_klassifikator.py       # Test der Merkmale
```

7.3 Erprobung ohne Roboter: Simulation (VRC)

Die Simulation benötigt Windows (hier Windows 11 in einer UTM-VM unter macOS) und das COBOTTA-Softwarepaket von DENSO (Ordner `Software`, Version 1.2.9). Die Lizenzschlüssel stellt der Betreuer bereit; sie gehören nicht in das Repository. Bewährte Reihenfolge:

1. **DENSO ROBOTICS License Manager** installieren (Ordner `LicenseManager`) und den WINCAPS-III-Schlüssel eintragen.
2. **VRC** installieren (Ordner `VRC`); den VRC-Schlüssel im Lizenzdialog des Installers eingeben. (Im zuvor installierten License Manager wurde er als ungültig abgelehnt.)
3. **WINCAPS III** aus dem vollständigen `Software`-Ordner installieren, da das Setup weitere Installationsdateien im selben Verzeichnis erwartet. Dabei wird auch ORiN2 mit dem bCapListener installiert. Das ORiN2-Setup allein meldete „License is invalid“.
4. `TP.exe` (VRC) starten und `Cobotta_test.WPJ` öffnen.
5. `bCapListener.exe` (unter `C:\ORiN2\CAO\ProviderLib\b-CAP`) starten und die Firewall-Freigabe bestätigen; das Fenster während der Versuche geöffnet lassen.
6. IP-Adresse der VM mit `ipconfig` ermitteln und vom Mac prüfen: `nc -vz <IP> 5007` muss „succeeded“ melden.
7. `python scripts/test_bcap_verbindung.py --ip <IP>`, danach `python main.py --roboter bcap --ip <IP>`.

7.4 Anbindung an einen realen COBOTTA

1. Rechner per Netzwerkkabel mit der Steuerung verbinden und eine feste IP-Adresse im Netz der Steuerung einstellen, z. B. 192.168.0.10, Subnetzmaske 255.255.255.0.
2. An der Steuerung die Ausführungsberechtigung (Executable Token) für Ethernet freigeben.
3. Arbeitsraumgrenzen in `config.yaml` an den Aufbau anpassen.
4. Falls ein elektrischer Greifer montiert ist: `greifer.vorhanden: true`.
5. `scripts/test_bcap_verbindung.py` ausführen. Das Skript verbindet sich, liest die Position, beobachtet sie 5 s lang und fährt erst nach Bestätigung einen Schritt von 5 mm in Z und zurück. Hand am Not-Halt behalten.
6. In `config.yaml` `roboter.backend: bcap` setzen und `main.py` starten.

7.5 Bedienung

Nach dem Start zeigt das Fenster das Kamerabild mit Zustand, aktiver Achse (farbig), Schrittweite, erkannter Geste, Position und letzter Meldung. Ablauf: Faust halten (Freigabe), mit dem Zeigefinger die Achse wählen, mit Daumen hoch/runter verfahren, mit dem I-love-you-Zeichen zwischen 5 mm und 1 mm umschalten, mit der offenen Hand stoppen. Die Taste `q` beendet das Programm; dabei wird der Roboter gestoppt und freigegeben.

8 Hinweise zur Weiterentwicklung

- **Andere Zuordnung der Gesten:** nur den Abschnitt `gesten` in `config.yaml` ändern; es stehen die sieben MediaPipe-Gesten zur Verfügung.

- **Neue Geste (eigener Klassifikator):** Geste in `KLASSEN` in `src/klassifikator.py` und `scripts/aufnahme_datensatz.py` ergänzen, alle Sitzungen neu aufnehmen, `scripts/training_klassifikator.py` ausführen und die Geste in `config.yaml` einem Befehl zuordnen.
- **Neuer Befehl:** in `src/controller.py` einen weiteren Zweig für den Befehl ergänzen, in `src/safety.py` die zugehörige Prüfung und Ausführung, und den Test in `scripts/test_steuerung.py` erweitern.
- **Anderer Roboter** (z. B. igus ReBeL): eine neue Klasse von `RobotInterface` ableiten, die sechs Methoden implementieren und in `src/robot/__init__.py` unter einem neuen Backend-Namen eintragen. Steuerung und Sicherheitsschicht bleiben unverändert.
- **Schwellen anpassen:** neue Aufnahmen mit `scripts/aufnahme_erkennung.py` erstellen, mit `scripts/analyse_erkennung.py` auswerten und die Werte in `config.yaml` begründet anpassen.

9 Fazit und Ausblick

Es wurde ein System entwickelt, mit dem ein DENSO COBOTTA berührungslos über sieben Handgesten verfahren werden kann. Das Bedienkonzept überträgt den Tippbetrieb des Teach Pendants auf Gesten. Das mehrstufige Sicherheitskonzept mit Freigabe, priorisiertem Stopp, Überwachung der Hand, Entprellung und unabhängiger Sicherheitsschicht wurde durch 21 automatisierte Prüfungen bestätigt. Die Gestenerkennung erreichte über 1416 Bilder und vier Aufnahmebedingungen eine Erkennungsrate von 97,0 %. Verwechslungen zwischen Gesten traten nicht auf, und bei freier Handbewegung wurde kein Bewegungsbefehl ausgelöst. Ein eigener, auf selbst aufgenommenen Daten trainierter Klassifikator (kNN mit 28 Formmerkmalen) erreichte auf ungesehenen Sitzungen einen Makro-F1 von 0,982 und war auf der Testsitzung mindestens gleichwertig zum MediaPipe-Modell. Dabei zeigte sich, dass eine Bewertung innerhalb derselben Aufnahmen die Güte überschätzt und dass Merkmale, die die Form der Hand beschreiben, deutlich besser verallgemeinern als die rohen Koordinaten. In der Simulation mit der virtuellen DENSO-Steuerung lief die vollständige Kette über b-CAP: 236 Schritte wurden exakt ausgeführt, der Stopp löste nach 35 ms aus, und alle Schritte über die Arbeitsraumgrenzen hinaus wurden abgelehnt.

Grenzen des Systems:

- Es wird nur die Position des Werkzeugs verändert, nicht seine Orientierung.
- Die Reaktionszeit wird durch die Bildrate der Kamera begrenzt (15 Bilder/s bei wenig Licht).
- Die Bewegung wird synchron ausgeführt; während eines Schritts wird kein neues Bild ausgewertet. Ein Stopp wirkt daher erst nach dem laufenden Schritt (in der Simulation bis 0,58 s bei 5 mm und 0,26 s bei 1 mm).
- Die Software-Sicherheitsfunktionen ersetzen keine zertifizierte Sicherheitstechnik.

Mögliche Erweiterungen:

- zusätzliche Gesten mit dem eigenen Klassifikator, z. B. zur Steuerung der Orientierung, sowie Aufnahmen weiterer Personen,
- redundante Stopp-Erkennung: Stopp, sobald MediaPipe *oder* das eigene Modell die offene Hand erkennt (zwei unabhängige Verfahren für die wichtigste Funktion),

- asynchrone Bewegung mit sofortigem Abbruch während eines Schritts,
- eine Kamera mit höherer Bildrate für kürzere Reaktionszeiten,
- kontinuierliches Verfahren, solange eine Geste gehalten wird.

Literatur

[1] Google: *MediaPipe Gesture Recognizer*. Online-Dokumentation, `https://developers.google.com/edge/mediapipe/solutions/vision/gesture_recognizer`

[2] DENSO WAVE: *orin_bcap – b-CAP-Client-Bibliotheken (C, C#, Java, Python)*. MIT-Lizenz, `https://github.com/DENSORobot/orin_bcap`

[3] OpenCV: *Open Source Computer Vision Library*. `https://opencv.org`

[4] Pedregosa, F. u. a.: *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research 12 (2011), S. 2825–2830. `https://scikit-learn.org`

A Konfigurationsdatei

```
# Gestensteuerung für den DENSO COBOTTA - Konfiguration
# Alle einstellbaren Werte stehen hier, damit der Code für einen neuen Aufbau
# nicht geändert werden muss.

kamera:
  index: 0              # 0 = eingebaute Webcam des MacBooks
  spiegeln: true        # Bild spiegeln (natürlicher beim Gestikulieren)

erkennung:
  backend: mediapipe    # mediapipe = vortrainierter Gesture Recognizer | eigen = eigener
    Klassifikator
  modell_pfad: models/gesture_recognizer.task       # MediaPipe-Modell (Hand + Handpunkte, immer
     benötigt)
  klassifikator_pfad: models/eigener_klassifikator.joblib   # erzeugt von scripts/
    training_klassifikator.py
  min_konfidenz: 0.55   # Gesten unterhalb dieser Konfidenz werden ignoriert (gemessen:
    Mittelwerte 0,61-0,90 je nach Geste)
  stopp_konfidenz: 0.45 # niedrigere Schwelle für STOPP: Fehlstopp ist harmlos, verpasster Stopp
     gefährlich (fail-safe)
  stabile_frames: 5     # gleiche Geste in N aufeinanderfolgenden Bildern, bevor ein Befehl
    ausgelöst wird (gemessen ca. 230-270 ms bei 15-17 Bildern/s)
  stopp_frames: 2       # STOPP reagiert schneller als alle anderen Gesten (gemessen ca. 70 ms)
  freigabe_frames: 15   # Faust so lange halten (gemessen ca. 0,9 s), um nach einem STOPP
    freizugeben
  sperrzeit_s: 0.5      # Pause nach einem Befehl, damit eine Geste = ein Schritt
  keine_hand_timeout_s: 1.0  # so lange keine Hand sichtbar -> STOPP

# Zuordnung Geste -> Befehl (Gestennamen von MediaPipe)
gesten:
  Open_Palm: STOPP
  Pointing_Up: ACHSE_WECHSELN
  Thumb_Up: SCHRITT_PLUS
  Thumb_Down: SCHRITT_MINUS
  Victory: GREIFER_UMSCHALTEN
  Closed_Fist: NEUTRAL      # Faust halten = Freigabe nach STOPP
  ILoveYou: SCHRITTWEITE_UMSCHALTEN   # Grob-/Feinschritt umschalten (wie Tippschrittweite am
    Teach Pendant)
```

```
bewegung:
  schrittweiten_mm: [5.0, 1.0]  # Grobschritt / Feinschritt; der erste Wert gilt beim Start
  geschwindigkeit_prozent: 10  # reduzierte Geschwindigkeit am realen Roboter
  max_schritt_mm: 10.0       # Sicherheitsgrenze: größerer Schritt wird abgelehnt
  bahnende: "@E"             # DENSO-Bewegungsoption: @E = warten, bis das Ziel erreicht ist (
    Encoder),
                             # @0 = bis der Sollwert das Ziel erreicht, @P = Überschleifen (
    kehrt früher zurück)

# Zulässiger Arbeitsraum des Werkzeugs in Roboter-Basiskoordinaten [mm].
# PLATZHALTERWERTE - vor dem Einsatz am realen COBOTTA prüfen!
arbeitsraum:
  x: [150.0, 300.0]
  y: [-150.0, 150.0]
  z: [50.0, 250.0]
  start: [200.0, 0.0, 150.0]

roboter:
  backend: mock         # mock | bcap
  ip: 192.168.0.1       # realer COBOTTA (aus Cobotta_test/Setting.xml); Simulation: IP der
    Windows-VM
  port: 5007            # b-CAP-Port
  provider: CaoProv.DENSO.VRC   # laut DENSO-Python-Beispiel für direkte b-CAP-Verbindung (real
    und Simulation)
  timeout_s: 3.0        # Wartezeit auf Antwort der Steuerung

greifer:
  vorhanden: false      # erst auf true setzen, wenn am COBOTTA ein elektrischer Greifer
    montiert ist
  offen_mm: 30.0        # Backenabstand offen
  geschlossen_mm: 5.0   # Backenabstand geschlossen
  geschwindigkeit_prozent: 50
```